

CYBERSECURITY INTERNSHIP

WEEK 4 ASSIGNMENT REPORT

Basic Network Setup and Initial Security Testing

Submitted by: Gunn Bhansali

Internship domain: Cybersecurity

Tools Used:

Vmware Workstation, Kali Linux, Ubuntu, Nmap , Wireshark

Submission Date:

14th September 2026

TABLE OF CONTENTS

1. Introduction to Network Security
2. Objectives of the Assignment
3. Lab Setup and Network Configuration
4. Services Installation and Verification
5. Network Scanning Using Nmap
6. Traffic Analysis Using Wireshark
7. Mini Project Workflow and Findings
8. Conclusion and Learning Outcomes
9. References

1. INTRODUCTION TO NETWORK SECURITY

Network security is the practice of protecting computer networks, systems, devices, and data from unauthorized access, misuse, attacks, and other security threats. It involves a combination of security technologies, policies, configurations, and monitoring techniques to maintain the confidentiality, integrity, and availability of network resources.

In modern organizations, networks are used to exchange sensitive information and provide access to important services. Therefore, securing network infrastructure is an essential part of cybersecurity. Common network security threats include unauthorized access, malware, denial-of-service attacks, packet interception, and exploitation of vulnerable services.

Network security testing helps security professionals understand how systems are exposed to potential threats. Tools such as Nmap can be used to discover open ports and identify services running on a target system, while Wireshark can be used to capture and analyze network traffic.

In this internship assignment, a small isolated laboratory network was created using VMware Workstation, with Kali Linux acting as the attacker/scanning machine and Ubuntu Linux acting as the target machine. Apache Web Server was configured on Ubuntu, Nmap was used for network scanning, and Wireshark was used to analyze HTTP, DNS, and ICMP traffic.

The purpose of this practical exercise was to understand basic network security concepts and demonstrate an initial security testing workflow in a controlled laboratory environment.

2. OBJECTIVES OF THE ASSIGNMENT

The main objective of this assignment was to understand the fundamentals of network security through a controlled practical laboratory environment. The assignment focused on setting up a small network, configuring a basic service, performing network reconnaissance, and analyzing network traffic.

The specific objectives were:

1. To create a basic network environment using VMware Workstation with Kali Linux and Ubuntu Linux.
2. To configure both virtual machines on the same network and verify their connectivity.
3. To install and configure the Apache web server on the Ubuntu target machine.
4. To verify that the configured web service was running correctly and accessible from the Kali machine.
5. To perform network scanning using Nmap and identify open ports and running services on the target.
6. To save the Nmap scan results for further analysis and documentation.
7. To capture network traffic using Wireshark.
8. To filter and analyze basic network protocols including HTTP, DNS, and ICMP.
9. To understand the complete workflow of basic network reconnaissance and traffic analysis in a controlled lab environment
10. To document the observations and findings obtained during the practical exercise

3. LAB SETUP AND NETWORK CONFIGURATION

A controlled virtual laboratory environment was created using VMware Workstation. Two virtual machines were used for the practical exercise: Kali Linux as the attacker/scanning machine and Ubuntu Linux as the target machine.

Both virtual machines were configured to use the same VMware VMnet1 Host-only network, allowing them to communicate with each other within the isolated laboratory network.

The network configuration used during the practical was:

Kali Linux (Attacker): 192.168.198.128/24

Ubuntu Linux (Target): 192.168.198.129/24

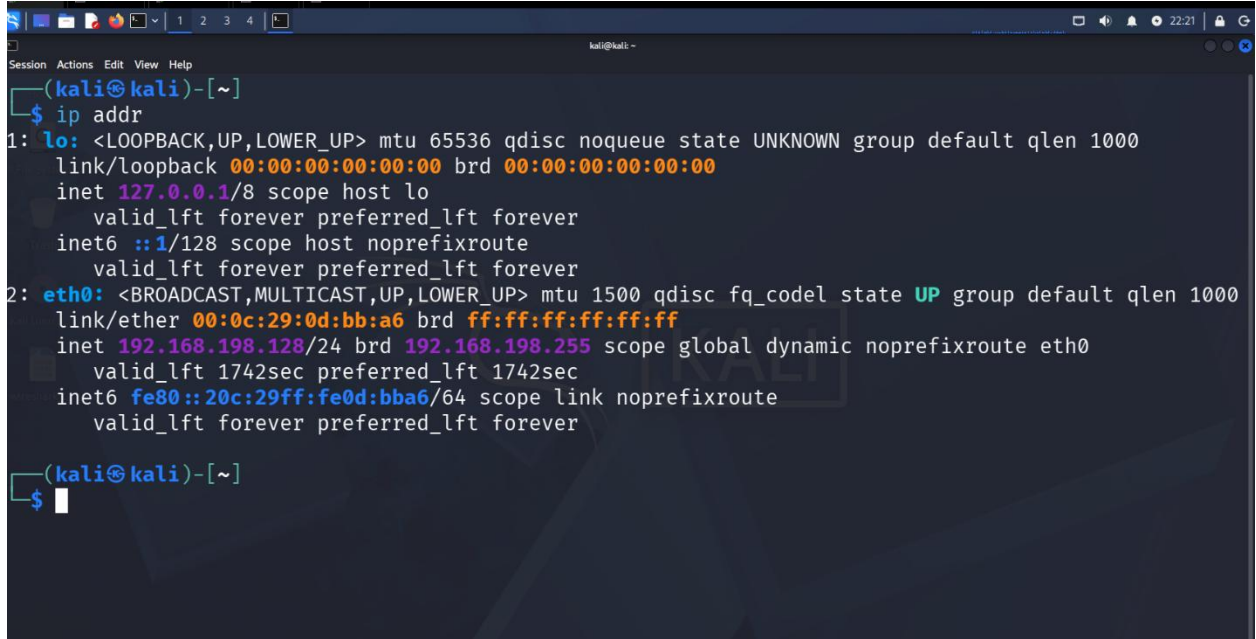
Network: 192.168.198.0/24

Virtual Network: VMnet1 (Host-only)

After configuring the network, connectivity between the two machines was verified using the ping command from Kali Linux to the Ubuntu target. Four ICMP echo requests were sent and all four were successfully received, confirming communication between the machines.

This setup provided a suitable controlled environment for performing the subsequent Apache service configuration, Nmap scanning, and Wireshark traffic analysis.

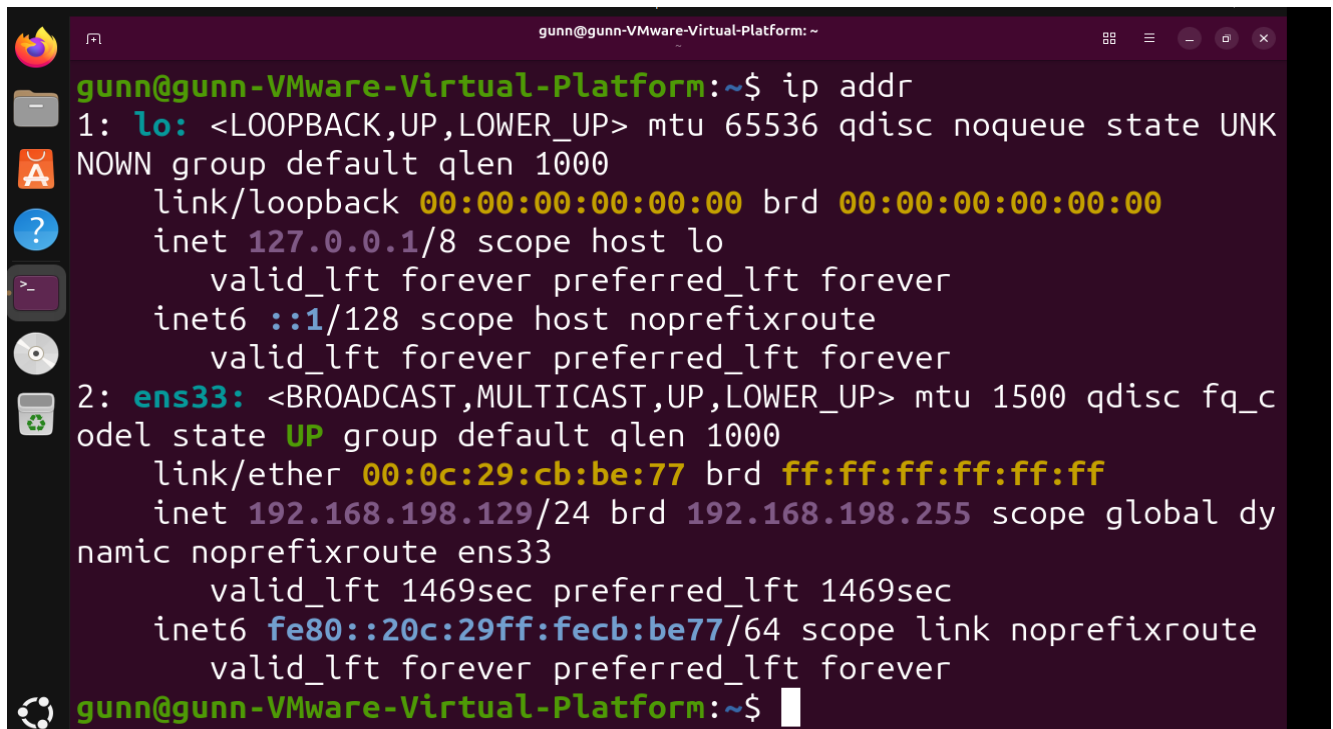
Figure 1: Kali Linux network configuration showing IP address



```
(kali@kali)-[~]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:0d:bb:a6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.198.128/24 brd 192.168.198.255 scope global dynamic noprefixroute eth0
        valid_lft 1742sec preferred_lft 1742sec
    inet6 fe80::20c:29ff:fe0d:bba6/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$
```

Figure 2: Ubuntu target network configuration showing IP address



```
gunn@gunn-VMware-Virtual-Platform:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKN
    OWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_c
    odel state UP group default qlen 1000
    link/ether 00:0c:29:cb:be:77 brd ff:ff:ff:ff:ff:ff
    inet 192.168.198.129/24 brd 192.168.198.255 scope global dy
    namic noprefixroute ens33
        valid_lft 1469sec preferred_lft 1469sec
    inet6 fe80::20c:29ff:fe0d:be77/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

gunn@gunn-VMware-Virtual-Platform:~$
```

4. SERVICES INSTALLATION AND VERIFICATION

For the target machine, an Apache web server was installed on Ubuntu Linux. Apache was selected as the basic web service for the laboratory because it provides a simple way to demonstrate service configuration, network accessibility, and security testing.

After installation, the Apache service was checked using the `systemctl` service-management command. The service status showed that Apache was active and running. The web server was also tested locally on the Ubuntu machine and returned an HTTP 200 OK response.

The service was then tested remotely from the Kali Linux machine using the Ubuntu target IP address, 192.168.198.129. The HTTP request successfully connected to port 80 and returned HTTP 200 OK, confirming that the Apache web server was accessible across the laboratory network.

FTP was not configured because it was listed as an optional service in the assignment.

Figure 3: Apache web server service active and running on Ubuntu

```
Stopped.
gunn@gunn-VMware-Virtual-Platform:~$ sudo systemctl restart apache2
gunn@gunn-VMware-Virtual-Platform:~$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enable
d)
   Active: active (running) since Tue 2026-09-08 22:56:00 IST; 28s ago
 Invocation: 2b6fd80b05174118825eb6cce06a57fd
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 19593 (apache2)
  Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes serve
d/sec:  0 B/sec"
    Tasks: 55 (limit: 1570)
   Memory: 5.6M (peak: 6M)
      CPU: 150ms
   CGroup: /system.slice/apache2.service
           └─19593 /usr/sbin/apache2 -k start -DFOREGROUND
             └─19601 /usr/sbin/apache2 -k start -DFOREGROUND
               └─19602 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 08 22:56:00 gunn-VMware-Virtual-Platform systemd[1]: Starting ap...
Sep 08 22:56:00 gunn-VMware-Virtual-Platform apachectl[19593]: AH005...e
Sep 08 22:56:00 gunn-VMware-Virtual-Platform systemd[1]: Started apa...
Hint: Some lines were ellipsized, use -l to show in full
```

Figure 4: Successful HTTP connection from Kali Linux to the Ubuntu Apache server

```
—(kali㉿kali)-[~]
—$ wget -O /tmp/test.html http://192.168.198.129/

--2026-09-11 11:27:32-- http://192.168.198.129/
Connecting to 192.168.198.129:80 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10672 (10K) [text/html]
Saving to: '/tmp/test.html'

/tmp/test.html          100%[=====>] 10.42K  --.-KB/s  in 0s

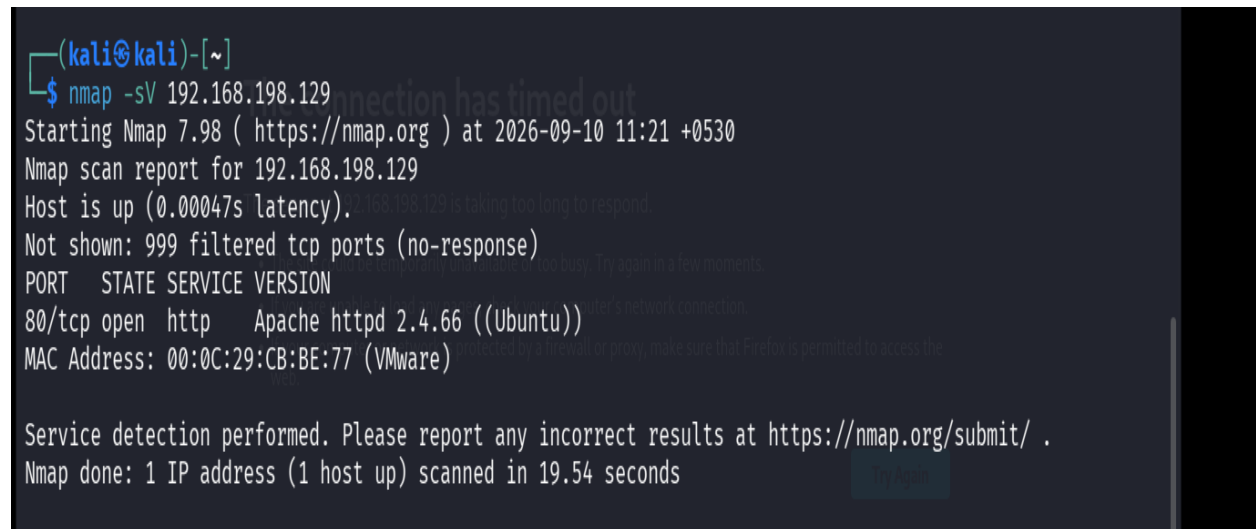
2026-09-11 11:27:32 (453 MB/s) - '/tmp/test.html' saved [10672/10672]

—(kali㉿kali)-[~]
—$
```

5. NETWORK SCANNING USING NMAP

Nmap was used from the Kali Linux attacker machine to scan the Ubuntu target machine at IP address 192.168.198.129. The purpose of the scan was to identify reachable ports and determine the services running on the target. The version-detection scan was performed using the -sV option. The scan identified the open port 80/tcp, which was running the HTTP service provided by Apache 2.4.66 on Ubuntu. The result confirmed that the target machine was reachable and that its Apache web service was exposed on the network. The scan output was saved in a text file named nmap_task3.txt for further analysis and submission.

Figure 5: Nmap service-version scan identifying the open HTTP port on the Ubuntu target



```
(kali㉿kali)-[~]
└─$ nmap -sV 192.168.198.129
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-10 11:21 +0530
Nmap scan report for 192.168.198.129
Host is up (0.00047s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.66 ((Ubuntu))
MAC Address: 00:0C:29:CB:BE:77 (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.54 seconds
```

6. TRAFFIC ANALYSIS USING WIRESHARK

Wireshark was used to capture and analyze network traffic generated within the laboratory environment. The capture was performed on the Kali Linux network interface, eth0, while communicating with the Ubuntu target machine.

During the traffic analysis, three basic protocols required by the assignment were examined: ICMP, HTTP, and DNS. Display filters were applied in Wireshark to isolate packets belonging to each protocol and understand the communication between the systems.

ICMP Analysis: ICMP traffic was generated by sending ping requests from Kali Linux to the Ubuntu target at 192.168.198.129. The Wireshark icmp filter displayed Echo Request and Echo Reply packets, confirming successful network communication between the two machines.

HTTP Analysis: HTTP traffic was generated by accessing the Apache web server running on the Ubuntu target. The http filter displayed the HTTP communication, including the HTTP GET request and the successful HTTP 200 OK response from the Apache server.

DNS Analysis: DNS packets were also observed during the traffic capture. The dns filter displayed DNS query packets using port 53, demonstrating DNS-related network communication in the captured traffic.

The captured traffic was saved as Task4_Traffic_Capture.pcapng for supporting evidence and further analysis.

Figure 6: ICMP Echo Request and Echo Reply packets captured in Wireshark

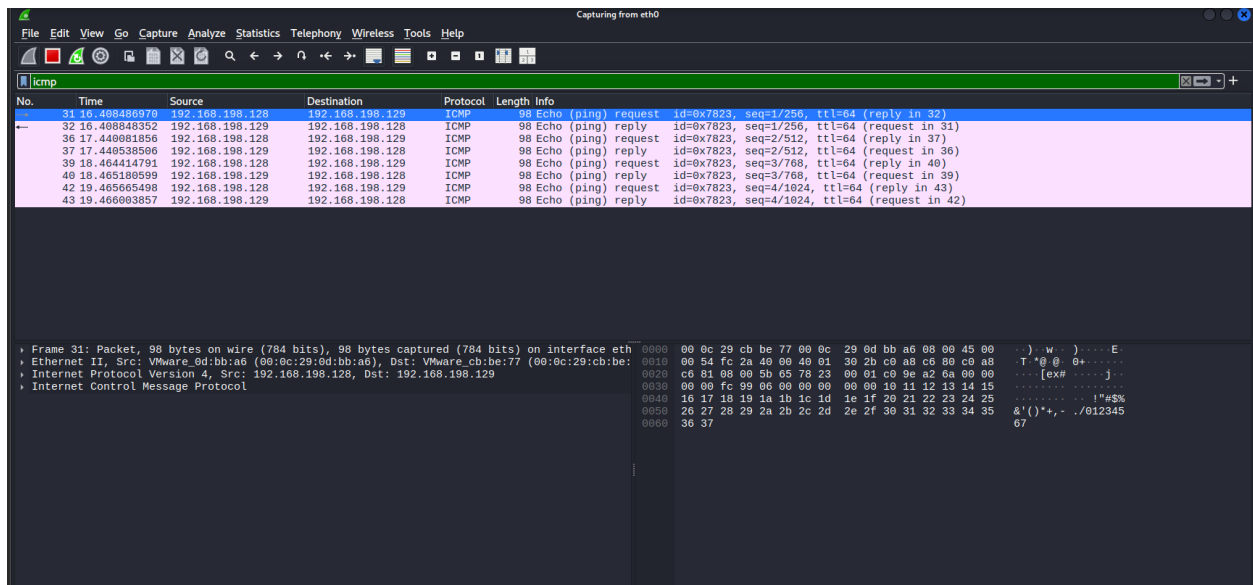


Figure 7: HTTP GET request and successful HTTP 200 OK response captured in Wireshark

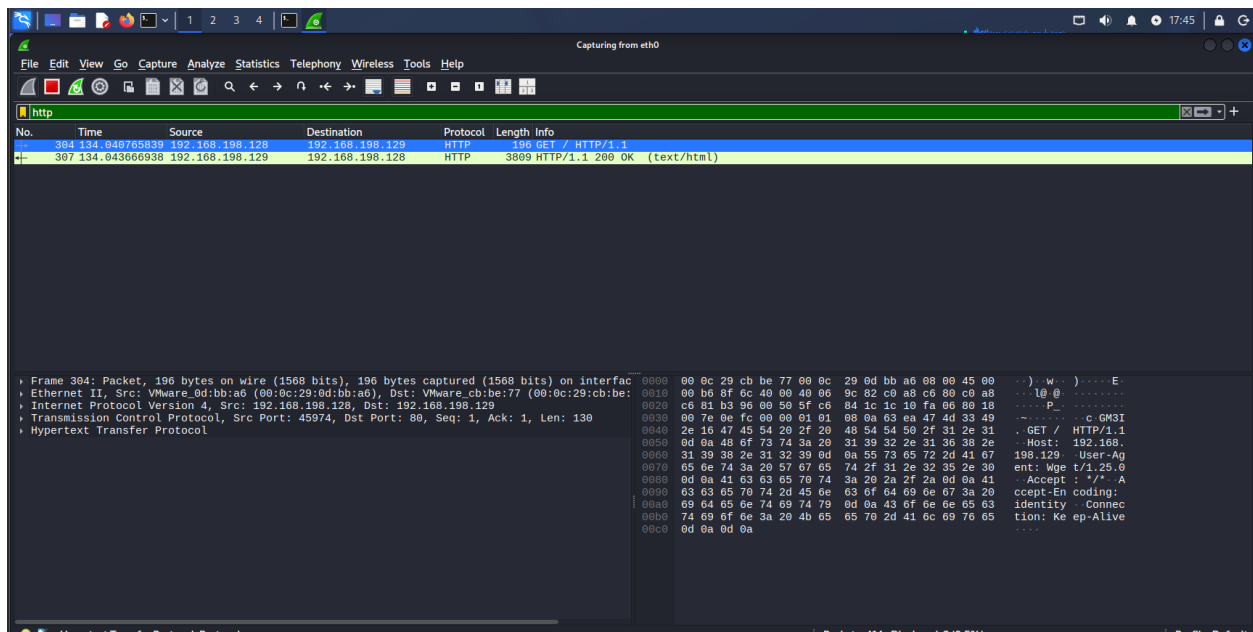


Figure 8: DNS query packets captured and analyzed in Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
537	241.748693911	192.168.198.128	192.168.198.1	DNS	96	Standard query 0x21b1 A ntp-bootstrap.ubuntu.com.localdomain
538	246.784818228	192.168.198.128	192.168.198.1	DNS	93	Standard query 0x9fe3 A 1.debian.pool.ntp.org.localdomain
539	246.784964554	192.168.198.128	192.168.198.1	DNS	93	Standard query 0x8bee AAAA 1.debian.pool.ntp.org.localdomain
540	246.995254629	192.168.198.128	192.168.198.1	DNS	96	Standard query 0x21b1 A ntp-bootstrap.ubuntu.com.localdomain
541	251.710322766	192.168.198.128	192.168.198.1	DNS	93	Standard query 0x9fe3 A 1.debian.pool.ntp.org.localdomain
542	251.710477739	192.168.198.128	192.168.198.1	DNS	93	Standard query 0x8bee AAAA 1.debian.pool.ntp.org.localdomain
544	256.716501764	192.168.198.128	192.168.198.1	DNS	81	Standard query 0xbdd9 A 2.debian.pool.ntp.org
545	256.716709312	192.168.198.128	192.168.198.1	DNS	81	Standard query 0x5f24 AAAA 2.debian.pool.ntp.org
546	261.717216863	192.168.198.128	192.168.198.1	DNS	81	Standard query 0xbdd9 A 2.debian.pool.ntp.org
547	261.717489952	192.168.198.128	192.168.198.1	DNS	81	Standard query 0x5f24 AAAA 2.debian.pool.ntp.org
554	266.723237652	192.168.198.128	192.168.198.1	DNS	93	Standard query 0xf919 A 2.debian.pool.ntp.org.localdomain
555	266.723395829	192.168.198.128	192.168.198.1	DNS	93	Standard query 0x6c15 AAAA 2.debian.pool.ntp.org.localdomain
560	271.728978338	192.168.198.128	192.168.198.1	DNS	93	Standard query 0xf919 A 2.debian.pool.ntp.org.localdomain
561	271.729161562	192.168.198.128	192.168.198.1	DNS	93	Standard query 0x6c15 AAAA 2.debian.pool.ntp.org.localdomain
569	276.735110987	192.168.198.128	192.168.198.1	DNS	81	Standard query 0xf26f A 3.debian.pool.ntp.org
570	276.735262548	192.168.198.128	192.168.198.1	DNS	81	Standard query 0x0d71 AAAA 3.debian.pool.ntp.org
575	281.740461488	192.168.198.128	192.168.198.1	DNS	81	Standard query 0xf26f A 3.debian.pool.ntp.org
576	281.740650967	192.168.198.128	192.168.198.1	DNS	81	Standard query 0x0d71 AAAA 3.debian.pool.ntp.org

Frame 299: Packet, 93 bytes on wire (744 bits), 93 bytes captured (744 bits) on interface eth0
Ethernet II, Src: VMware_0d:bb:a6 (00:0c:29:0d:bb:a6), Dst: VMware_c0:00:01 (00:50:56:c0:00:01)
Internet Protocol Version 4, Src: 192.168.198.128, Dst: 192.168.198.1
User Datagram Protocol, Src Port: 52760, Dst Port: 53
Domain Name System (query)

0000 00 50 56 c0 00 01 00 0c 29 0d bb a6 00 00 45 00 PV).....E
0010 00 4f 90 30 40 00 40 11 93 9a c0 a8 c6 80 c0 a8 0 00 @
0020 c6 01 ce 18 00 35 00 3b 0e 20 e1 61 01 00 00 015 ; .a....
0030 00 00 00 00 00 01 33 0e 64 65 62 69 61 6e 643 .debian
0040 70 6f 6f 6c 03 6e 74 70 93 6f 72 67 0b 6c 6f 63 pool ntp org loc
0050 61 6c 64 6f 6d 61 69 6e 00 00 1c 00 01 aldomain

Domain Name System: Protocol | Packets: 576 - Displayed: 355 (61.6%) | Profile: Default

7. MINI PROJECT WORKFLOW AND FINDINGS

The mini project demonstrated a complete basic network security testing workflow in a controlled VMware laboratory environment. The workflow started with configuring Kali Linux and Ubuntu Linux on the same VMnet1 Host-only network. Connectivity between the two systems was verified successfully.

Next, the Apache web server was installed and configured on the Ubuntu target machine. The Apache service was confirmed to be active and running. A remote HTTP request from Kali Linux to the Ubuntu server returned an HTTP 200 OK response, confirming that the web service was accessible across the lab network.

The Ubuntu target was then scanned from Kali Linux using Nmap. The scan identified the open HTTP port and the Apache service running on the target. The scan results were saved in the `nmap_task3.txt` file.

Finally, Wireshark was used to capture network traffic and analyze basic protocols. ICMP Echo Request/Reply packets were observed during connectivity testing, HTTP packets were observed during communication with the Apache server, and DNS packets were identified using the DNS display filter.

The practical exercise demonstrated how network configuration, service identification, network scanning, and packet analysis can be combined to perform an initial security assessment of a system in an authorized laboratory environment.

8. CONCLUSION AND LEARNING OUTCOMES

This assignment provided practical exposure to the fundamentals of network security and basic security testing in a controlled virtual laboratory environment. A network consisting of Kali Linux and Ubuntu Linux was successfully configured using VMware Workstation and a Host-only network.

The Apache web server was installed and verified on the Ubuntu target machine. Nmap was then used to identify the open port and running service on the target. The scan results were saved for documentation and further analysis.

Wireshark provided practical experience in capturing and analyzing network packets. HTTP, DNS, and ICMP traffic were filtered and examined to understand how different protocols operate during network communication.

Through this exercise, I learned the importance of network configuration, service verification, network reconnaissance, and packet analysis in cybersecurity. I also gained practical familiarity with tools such as Nmap and Wireshark, which are commonly used for network assessment and traffic analysis.

Overall, the project helped establish a foundation for understanding how security professionals identify exposed services and analyze network communication in an authorized environment. The practical knowledge gained from this assignment can be further developed through advanced network scanning, vulnerability assessment, and security monitoring techniques.

9. REFERENCES

1. Nmap Documentation — <https://nmap.org/docs.html>
2. Wireshark Documentation — <https://www.wireshark.org/docs/>
3. Apache HTTP Server Documentation — <https://httpd.apache.org/docs/>
4. Ubuntu Documentation — <https://documentation.ubuntu.com/>
5. VMware Workstation Documentation — VMware, Inc.